FREELANCER WORK MANAGEMENT

Web & Mobile Application

Technical Blueprint & Development Roadmap

Stack	AI	Phase	Version
React + RN	**GPT-4**	**Full MVP**	**1.0.0**

1. Project Overview

This document provides a complete technical blueprint for building a Freelancer Work Management application — a full-stack platform designed to help business development and tech professionals maximize their freelancing success through intelligent automation, AI-powered analysis, and streamlined proposal workflows.

Primary Objective

Build a cross-platform (Web + Mobile) application that improves bidding efficiency, increases proposal success rate, automates research and timing decisions, and standardizes the proposal writing workflow — all powered by OpenAI GPT-4.

1.1 Core Modules

Module	Description
Auth & Users	Secure login, Google OAuth, profile management
Timezone Alerts	Smart notifications based on project posting patterns by region
Proposal Builder	Template library with greeting, strategy, closing, and P.S. sections
AI Analysis	GPT-4 powered project analysis, bid strategy, tech fit scoring
Profile Analysis	Screenshot upload, profile strength scoring and improvement tips
Records & Tracking	7-day proposal archive, submission logs, effectiveness tracking
File Storage	Document/image attachments with auto-expiry policies
Analytics Dashboard	Success rates, timing heatmaps, strategy comparisons

2. Technology Stack

The stack is chosen to maximize code reuse between web and mobile while ensuring scalability, developer velocity, and AI integration quality.

2.1 Frontend

Technology	Category	Purpose
React 18 (Web)	UI Framework	Vite + TypeScript, component-based SPA
React Native (Mobile)	Mobile Framework	Expo managed workflow for iOS & Android
React Navigation	Mobile Routing	Stack + Tab navigators
React Router v6	Web Routing	Client-side routing with protected routes
Zustand	State Management	Lightweight, shared logic across web/mobile
TanStack Query	Data Fetching	Caching, background sync, optimistic updates
Tailwind CSS	Web Styling	Utility-first, responsive design system
NativeWind	Mobile Styling	Tailwind classes for React Native
React Hook Form	Forms	Performant form management + Zod validation
Recharts	Analytics Charts	Composable chart library for dashboard

2.2 Backend

Technology	Category	Purpose
Node.js + Express	Runtime & Framework	Fast, non-blocking REST API server
TypeScript	Language	Type safety across the entire codebase
PostgreSQL	Primary Database	Relational data: users, proposals, records
Redis	Cache & Sessions	Session management, alert queues, rate limiting
Prisma ORM	DB Layer	Type-safe database queries, migrations

OpenAI SDK	AI Integration	GPT-4 API for analysis, recommendations
Firebase Admin	Push Notifications	FCM for mobile + web push alerts
node-cron	Scheduler	Timezone-aware cron jobs for alert system
Multer + Sharp	File Handling	Upload, compress and resize images
AWS S3	Object Storage	Secure file storage with lifecycle rules
JWT + Passport.js	Auth	JWT tokens, Google OAuth2 strategy
Zod	Validation	Runtime schema validation on all endpoints

2.3 Infrastructure & DevOps

Technology	Category	Purpose
Railway / Render	App Hosting	Backend API deployment with auto-scaling
Vercel	Web Frontend	CDN-powered React app deployment
Expo EAS	Mobile Build	Cloud builds + OTA updates for React Native
Supabase / Neon	Managed Postgres	Serverless PostgreSQL with connection pooling
Upstash Redis	Managed Redis	Serverless Redis for sessions and caching
GitHub Actions	CI/CD	Automated test, build, and deploy pipelines
Sentry	Error Monitoring	Real-time error tracking for web and mobile
Postman	API Docs	Automated API documentation and testing

3. System Architecture

The application follows a clean three-tier architecture with a shared backend serving both the React web app and the React Native mobile app. All business logic lives in the API layer, keeping clients thin and consistent.

Architecture Pattern

Client-Server with RESTful API | JWT-based stateless auth | Redis-backed session layer | Event-driven alert engine using cron + FCM | AI as a service layer via OpenAI SDK

3.1 High-Level Architecture Flow

Layer	Flow
Web Client (React)	Vite SPA → Nginx → API Gateway → Express
Mobile Client (RN)	Expo App → HTTPS → Same Express API
API Layer	Express + TypeScript → Prisma → PostgreSQL
AI Layer	Express service → OpenAI SDK → GPT-4 API
Alert Engine	node-cron → Redis Queue → FCM → Device
File Pipeline	Multer → Sharp (compress) → S3 → Signed URL
Auth Flow	Request → JWT verify → Passport → User context
Cache Layer	Redis TTL cache on AI responses + DB queries

3.2 API Structure

All endpoints follow RESTful conventions under the base path /api/v1. All protected routes require a valid JWT Bearer token in the Authorization header.

Endpoint Group	Routes
/api/v1/auth	POST /signup, /login, /google, /logout, /refresh
/api/v1/users	GET+PUT /profile, GET /me, POST /screenshot
/api/v1/templates	CRUD for proposal templates and components
/api/v1/proposals	CRUD proposals, GET /history, POST /generate
/api/v1/ai	POST /analyze, /strategy, /profile-review, /bid-amount
/api/v1/alerts	GET/PUT alert preferences, GET /timezones
/api/v1/records	GET proposal records, stats, timeline data

/api/v1/analytics	GET dashboard metrics, heatmaps, comparisons
/api/v1/files	POST /upload, GET /signed-url, DELETE /file

4. Database Schema Design

PostgreSQL is the primary datastore. Tables are designed for normalized data with appropriate indexes for performance. All timestamps use UTC. Proposal records use a soft-delete with a 7-day TTL enforced by a cron job.

4.1 Core Tables

Table	Key Columns	Purpose
users	id, email, name, avatar_url, google_id, timezone, created_at	Core user identity
user_profiles	id, user_id, skills[], experience, bio, hourly_rate, platforms[]	Professional details
proposal_templates	id, user_id, name, category, strategy, components (JSONB)	Template library
template_components	id, user_id, type (greeting/opening/strategy/closing/ps), content, tags[]	Reusable snippets
proposals	id, user_id, project_id, content, status, created_at, expires_at	Submitted proposals (7d TTL)
project_records	id, user_id, title, description, client_country, timezone, tech_stack[], bid_amount	Project metadata
ai_analyses	id, user_id, project_id, analysis (JSONB), strategy, effort_level, bid_range	GPT-4 outputs
alert_configs	id, user_id, countries[], timezones[], active_hours, notification_channels	Alert preferences
files	id, user_id, proposal_id, s3_key, filename, mime_type, expires_at	Attachments (7d TTL)
analytics_events	id, user_id, event_type, metadata (JSONB), created_at	Audit + analytics trail

4.2 Key Design Decisions

- JSONB columns for flexible AI analysis output and template components — avoids over-normalization for semi-structured data
- expires_at on proposals and files — cron job runs daily at midnight UTC to purge records older than 7 days
- analytics_events as append-only event log — enables flexible aggregation queries for dashboard without altering core records
- Composite indexes on (user_id, created_at) across proposals and records for fast user-scoped queries

- Redis stores JWT refresh tokens with TTL matching token expiry, plus AI response cache (1hr TTL) to reduce OpenAI API costs

5. Module Specifications

5.1 Authentication & User Management

Auth Strategy

JWT access tokens (15min) + Refresh tokens (7 days) stored in Redis. Google OAuth 2.0 via Passport.js. Passwords hashed with bcrypt (12 rounds). All auth endpoints rate-limited at 10 req/min per IP.

- Signup: email/password or Google → create user record → issue JWT pair → return tokens
- Login: validate credentials → check refresh token pool → issue new JWT pair
- Google Sign-In: OAuth2 code flow → fetch Google profile → upsert user → issue JWT
- Profile: editable fields: name, avatar, bio, skills, timezone, hourly rate, platforms
- Session: access token in memory (web) / SecureStore (mobile) → refresh on 401 response

5.2 Timezone-Based Alert System

Alert Logic

Projects on platforms like Upwork are typically posted by clients in their business hours (9AM-6PM local). The system pre-calculates optimal bidding windows per region and schedules push notifications ~15 minutes before peak project-posting times.

- Users select target countries/regions they bid on (e.g., USA, UK, Australia)
- System maps each country to its primary timezone and calculates UTC offset
- node-cron jobs fire at calculated UTC times corresponding to 8:45 AM in each target timezone
- Notification payload includes: country, estimated project volume level, suggested action
- Users can set do-not-disturb hours and enable/disable per-country alerts independently
- Firebase Cloud Messaging delivers push to both iOS/Android and web browsers

5.3 Proposal Builder & Template System

The proposal builder uses a component-based assembly model. Each proposal is constructed from modular blocks that can be saved, reused, and tagged.

Component	Description
Greeting	e.g., 'Hi [Name],' or 'Hello,' — personalized openers
Opening Statement	Hook line referencing client's problem or project goal

Strategy Content	Main body: approach, methodology, relevant experience
Closing Line	Confident call-to-action or invitation to discuss
Regards	Professional sign-off with name and title
P.S. Line	Bonus info, social proof, or urgency booster

- Templates can be tagged by strategy (e.g., 'Fixed Price', 'Agency Style', 'Concise Punch')
- Reference library: store past successful proposals for inspiration — not editable, read-only
- Quick compose: select one component from each category → auto-assembled proposal preview
- Proposal preview renders as formatted text with copy-to-clipboard functionality

5.4 AI-Based Project Analysis (GPT-4)

AI Integration

Each project description is sent to GPT-4 with a structured system prompt that instructs the model to return a JSON analysis object. Responses are cached in Redis for 1 hour by a hash of the project description to reduce API costs for identical/similar projects.

Analysis Output	Details
Proposal Structure	Recommended sections and emphasis based on project type
Bidding Strategy	Fixed price vs. hourly, competitive vs. premium positioning
Effort Estimation	Low / Medium / High with rough hour breakdown
Technology Fit	Match score between your skills and project requirements
Bid Amount Range	Min/max bid suggestion based on effort + market positioning
Red Flags	Unclear specs, low budget signals, vague requirements warnings
Winning Angle	Unique selling point to highlight in the proposal

GPT-4 Prompt Design:

- System prompt: defines role as 'expert freelance consultant and bid strategist'
- User prompt: structured template injecting project description + user's skill profile
- Output format: strict JSON schema enforced via function calling / structured output mode
- Fallback: if structured parse fails, retry once with explicit JSON instruction

5.5 Freelancer Profile Analysis

- User uploads a screenshot of their freelancer profile (Upwork, Fiverr, Freelancer.com, etc.)
- Image is sent to GPT-4 Vision with a profile review prompt

- AI evaluates: headline clarity, skills relevance, bio quality, portfolio presence, completeness score
- Output: profile strength score (0-100), top 3 improvement areas, estimated impact of each
- Effort estimate: days/weeks to implement suggestions based on complexity
- Previous analyses are stored per user so progress can be tracked over time

5.6 Proposal & Project Records

- Every submitted proposal is stored with: project title, description, client country, timezone, tech stack, bid amount, timestamp, and proposal text
- Records are retained for exactly 7 days — a nightly cron job at 00:00 UTC purges expired records
- Users can manually mark a proposal as Won, Lost, Pending, or No Response
- Effectiveness tracking: win rate per technology, per client region, per proposal strategy
- Export: users can export their records as CSV before they expire

6. UI / UX Structure

6.1 Web Navigation (Sidebar Layout)

Route	Screen Content
/ (Dashboard)	Analytics overview, quick stats, recent activity
/proposals	Proposal list, create new, search and filter
/builder	Component assembly UI, template picker, live preview
/templates	Template library CRUD, organize by category/strategy
/ai-analyze	Paste project description → get AI analysis report
/records	Proposal history table with filters, status tags
/alerts	Timezone/country selection, notification schedule
/profile	Edit profile, upload screenshot, view AI review
/settings	Account, notifications, data management

6.2 Mobile Navigation (Bottom Tab Layout)

Tab	Content
Home Tab	Dashboard cards: today's alerts, quick stats, recent proposals
Analyze Tab	Type or paste project → AI analysis results screen
Builder Tab	Quick proposal assembly → copy to clipboard
Records Tab	Swipeable proposal history list with status badges
Profile Tab	User profile, notifications, app settings

6.3 Design System

Token	Value & Usage
Primary Color	#1A56DB (Brand Blue) — CTAs, headers, active states
Dark	#1E293B — body text, card titles
Accent	#0EA5E9 — highlights, links, secondary actions
Success	#10B981 Green — won, positive metrics
Warning	#F59E0B Amber — pending, medium effort
Danger	#EF4444 Red — lost, high effort, red flags

Typography	Inter (Web) / System font (Mobile) — 14px base, 1.5 line height
Components	Shared component library via React Native Web for max reuse

7. Project Folder Structure

7.1 Monorepo Layout

Repository Structure

Use a monorepo (pnpm workspaces or Turborepo) to share types, utilities and constants between web, mobile and backend packages — reducing duplication and keeping the codebase consistent.

Path	Purpose
apps/web/	React + Vite web application
apps/mobile/	Expo React Native mobile app
apps/api/	Node.js Express backend server
packages/shared/	Shared TypeScript types, Zod schemas, constants, utils
packages/ui/	Shared React components (React Native Web compatible)
packages/config/	Shared ESLint, TypeScript, Tailwind configs
.github/workflows/	CI/CD pipelines for test + deploy
docker-compose.yml	Local dev: Postgres + Redis containers
turbo.json	Turborepo pipeline config for parallel builds

7.2 Backend Structure (apps/api/)

Directory	Contents
src/routes/	Express route handlers per module (auth, proposals, ai, etc.)
src/controllers/	Request/response logic, thin layer calling services
src/services/	Business logic (proposalService, aiService, alertService)
src/middleware/	auth.ts, rateLimit.ts, errorHandler.ts, upload.ts
src/lib/	openai.ts, redis.ts, s3.ts, firebase.ts clients
src/prisma/	schema.prisma + migrations
src/jobs/	Cron job definitions (expiry, alerts)
src/types/	Shared TS interfaces for API request/response
src/utils/	timezone.ts, pagination.ts, hashing.ts helpers

8. AI Prompt Engineering Strategy

Well-designed prompts are critical for consistent, structured AI outputs. All prompts use OpenAI's function calling / structured output feature to enforce JSON response schemas, eliminating brittle text parsing.

8.1 Project Analysis Prompt

System Prompt

You are an expert freelance consultant and proposal strategist with 10+ years of experience on Upwork, Toptal, and Freelancer.com. Analyze the given project and return a structured JSON assessment that helps the freelancer craft a winning proposal.

User prompt injects:

- Project title and full description
- Client country and posted timezone
- User's skills array and experience summary
- User's preferred hourly rate range

Structured output schema enforces:

- recommended_structure: string[] — ordered proposal sections
- bidding_strategy: 'fixed' | 'hourly' | 'milestone'
- effort_level: 'low' | 'medium' | 'high' + hours_estimate: number
- tech_fit_score: 0-100 + matched_skills: string[]
- bid_range: { min: number, max: number, currency: 'USD' }
- red_flags: string[] — potential risks identified
- winning_angle: string — single best unique selling point

8.2 Profile Review Prompt

Vision Model

Attach the profile screenshot as a base64 image. Use GPT-4 Vision (gpt-4o). System prompt instructs the model to evaluate the freelancer profile across 5 dimensions and return a structured improvement report.

- Dimensions scored 0-20 each: Headline (20), Bio (20), Skills (20), Portfolio (20), Completeness (20)
- Output: overall_score, dimension_scores, top_3_improvements[], effort_per_improvement
- Improvement items include: action, expected_impact ('high'/'medium'/'low'), estimated_days

8.3 Proposal Generation Assist

- User selects a template structure + AI analysis recommendations as input
- GPT-4 generates a draft proposal populating the selected components
- User can regenerate individual sections without rewriting the whole proposal
- AI suggestions appear as inline ghost text — user accepts or dismisses each

9. Security Measures

Area	Implementation
Authentication	JWT RS256 signed access tokens (15min) + Redis-backed refresh tokens (7d)
Password Storage	bcrypt with 12 rounds — no plain text passwords ever stored
API Security	Helmet.js headers, CORS whitelist, rate limiting per IP + per user
Input Validation	Zod schemas on all request bodies — reject any unexpected fields
File Uploads	MIME type whitelist (jpg/png/pdf), max 10MB, virus scan pre-S3 upload
S3 Access	All files accessed via short-lived (15min) presigned URLs — no public buckets
SQL Injection	Prisma ORM with parameterized queries — no raw SQL from user input
XSS Prevention	React auto-escaping + DOMPurify on any HTML rendering + CSP headers
OpenAI Keys	API keys only on server — never exposed to client. Prompt injection mitigation via input sanitization
Data Retention	Automatic 7-day TTL on proposals/files enforced at DB + S3 lifecycle level

10. Development Roadmap

The build is organized into 4 phases with all MVP features shipped in Phases 1-3. Phase 4 covers advanced features and scaling optimizations.

Phase	Duration	Priority	Modules
Phase 1	Weeks 1-3	Critical	Auth, User Profile, DB Schema, API Scaffold
Phase 2	Weeks 4-6	Critical	Proposal Builder, Templates, Records Management
Phase 3	Weeks 7-9	Critical	AI Analysis, Profile Review, Alert Engine
Phase 4	Weeks 10-11	High	Analytics Dashboard, File Storage, Performance
Phase 5	Weeks 12-13	Medium	Mobile Polish, Push Notifications, Beta Testing
Phase 6	Week 14+	Low	Export, Advanced Analytics, Performance Optimization

Phase 1 — Foundation (Weeks 1–3)

- Monorepo setup: Turborepo, pnpm workspaces, shared TypeScript config
- Backend: Express + Prisma + PostgreSQL schema + Redis connection
- Auth module: signup/login, Google OAuth, JWT + refresh token flow
- User profile API: create, read, update profile + skills
- React web app scaffold: routing, layout, auth context, protected routes
- React Native scaffold: Expo setup, navigation, auth screens
- Deploy skeleton: Railway (API), Vercel (web), Expo Go (mobile)

Phase 2 — Core Features (Weeks 4–6)

- Template component CRUD: create greeting, opening, strategy, closing, P.S. components
- Proposal builder UI: drag-and-drop assembly, live preview, copy button
- Template library: categorize by strategy, search and filter
- Proposal submission records: create, list, status updates, 7-day expiry logic
- Reference library: save successful proposals as read-only references
- Cron job: nightly expiry purge for proposals and files

Phase 3 — Intelligence Layer (Weeks 7–9)

- OpenAI service: GPT-4 integration, structured output, Redis cache
- Project analysis endpoint: full analysis JSON from project description
- Profile review: GPT-4 Vision integration for screenshot analysis
- Proposal generation assist: AI-populated draft from templates + analysis
- Alert system: timezone config UI, node-cron scheduler, FCM integration
- Push notifications: web push (service worker) + mobile FCM setup

Phase 4 — Analytics & Files (Weeks 10–11)

- Analytics dashboard: win rate, timing heatmap, strategy comparison charts
- File upload: S3 integration, presigned URLs, MIME validation
- Attach files to proposals: link documents/images to specific submissions
- Performance analytics: per-technology, per-region win rate breakdowns
- Data export: CSV download of proposal records before expiry

11. Cost & API Estimates

Monthly cost estimates for a solo user with moderate usage (20 analyses/day):

Service	Estimated Cost
OpenAI GPT-4o	~$15-30/month @ 20 project analyses/day + caching
Railway / Render	~$10-20/month for hobby/starter tier API + Postgres
Upstash Redis	Free tier (10k req/day) → $10/month if exceeded
AWS S3	~$2-5/month for file storage (7-day retention keeps costs minimal)
Expo EAS Build	Free tier (30 builds/month) → sufficient for indie use
Firebase FCM	Free (1M notifications/month included)
Vercel Web Hosting	Free hobby tier sufficient for personal use
Total Estimate	~$30-70/month depending on OpenAI usage volume

Cost optimization tips:

- Cache AI responses in Redis with 1-hour TTL — repeated analysis of same project hits cache, not OpenAI
- Use GPT-4o-mini for low-complexity tasks (e.g., component suggestions) and GPT-4o only for full analysis
- Batch similar timezone alerts to reduce FCM calls
- S3 lifecycle rules automatically delete files after 7 days — no manual cleanup needed

12. Getting Started — Quick Setup

12.1 Prerequisites

Requirement	Notes
Node.js	v20+ (LTS recommended)
pnpm	v8+ for monorepo workspace management
Docker Desktop	For local Postgres + Redis via docker-compose
Expo Go App	On your phone for mobile development
OpenAI API Key	From platform.openai.com — GPT-4o access required
Google OAuth Credentials	From Google Cloud Console
Firebase Project	For push notification credentials (FCM)

12.2 Initial Setup Steps

1. Clone repo and run pnpm install at monorepo root
2. Copy .env.example to .env in apps/api/ and fill all keys
3. Run docker-compose up -d to start Postgres + Redis locally
4. Run pnpm prisma migrate dev in apps/api/ to create DB schema
5. Run pnpm dev to start all apps in parallel via Turborepo
6. Open http://localhost:5173 for web, Expo Go for mobile

12.3 Key Environment Variables

Variable	Description
DATABASE_URL	PostgreSQL connection string
REDIS_URL	Redis connection string
JWT_SECRET	Long random string for JWT signing (RS256 preferred)
OPENAI_API_KEY	Your OpenAI API key with GPT-4o access
GOOGLE_CLIENT_ID	Google OAuth 2.0 client ID
GOOGLE_CLIENT_SECRET	Google OAuth 2.0 client secret
AWS_ACCESS_KEY_ID	AWS IAM user key for S3 access
AWS_SECRET_ACCESS_KEY	AWS IAM secret for S3 access
S3_BUCKET_NAME	Your S3 bucket name (private, no public access)

FIREBASE_SERVICE_ACCOUNT	JSON credentials for Firebase Admin SDK